A10 Deployment

Git folders, Databricks Asset Bundles, parameterised jobs and CI.

A10-L1 Git folders, one branch per change, and no secrets in the repository

Code flows from a branch, through review and CI, to production. Secrets flow from a secret store at run time, never through Git.

```python
api_key = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

Where do developers work in Databricks Git folders?	Their own clone under /Workspace/Users/<email>/, on their own branch.
Who updates production Git folders?	Only automation, after a pull request merges into the deployment branch.
Read a secret in a notebook	dbutils.secrets.get(scope="...", key="..."). Output shows [REDACTED].
A token was committed, then removed. Is it safe?	No. Git history keeps it. Rotate the token.

A10-L2 Declarative Automation Bundles and parameterised jobs

One reviewed file describes the job for every environment. The target decides where it writes.

```yaml
bundle:
  name: research_pipelines

variables:
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev nat_silver_pipeline
```

What are Declarative Automation Bundles?	Jobs and their settings as YAML in Git (formerly Databricks Asset Bundles). Same CLI: databricks bundle.
Targets and variables	Targets are environments. Variables hold what differs, such as the catalog, set per target.
What does mode: development do?	Prefixes resources with [dev <user>] and pauses schedules and triggers.
Read a job parameter in a notebook	dbutils.widgets.get("name"). Validate it before using it.

A10-L3 CI: lint, type-check, test and validate on every pull request

Every pull request is linted, typed, tested and validated. Only a merge to main deploys, as a service principal, with nothing secret in the file. The CLI version shown is an example: pin the version you tested.

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Four CI checks for a pipeline repository	ruff (lint and format), mypy, pytest with local Spark, databricks bundle validate.
Who deploys to production?	CI, after a merge to main, as a service principal.
OAuth M2M environment variables	DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET from the CI secret store.
Two ways CI lies	continue-on-error: true, and command || true. Both turn failures green.